Università di Genova

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

First Assignment

Equivalent representations of orientation matrices

Authors:

Nogarin Giacomo
Spluga Matteo
Tozzola Michele

Student IDs:

s8654515
s9185902
s8995647

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

November 11, 2025

Contents

1 Assignment description **3**
1.1 Exercise 1 - Angle-Axis to Rotation Matrix . . . 3
1.2 Exercise 2 - Rotation Matrix to Angle-Axis . . . 3
1.3 Exercise 3 - Euler Angles to Rotation Matrix . . . 4
1.4 Exercise 4 - Rotation Matrix to Euler Angles . . . 4
1.5 Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors . . . 4
1.6 Exercise 6 - Frame tree . . . 5

2 Exercise 1 **6**
2.1 Q1.1 . . . 6
2.2 Q1.2 . . . 6
2.3 Q1.3 . . . 6
2.4 Q1.4 . . . 6

3 Exercise 2 **6**
3.1 Q2.1 . . . 7
3.2 Q2.2 . . . 7
3.3 Q2.3 . . . 7
3.4 Q2.4 . . . 7
3.5 Q2.5 . . . 7
3.6 Q2.6 . . . 8

4 Exercise 3 **8**
4.1 Q3.1 . . . 8
4.2 Q3.2 . . . 8
4.3 Q3.3 . . . 8
4.4 Q3.4 - Q3.5 . . . 9

5 Exercise 4 **9**
5.1 Q4.1 . . . 9
5.2 Q4.2 . . . 9
5.3 Q4.3 . . . 9
5.4 Q4.4 . . . 10

6 Exercise 5 **10**
6.1 Q5.1 . . . 10
6.2 Q5.2 . . . 10

7 Exercise 6 **11**
7.1 Q6.1 . . . 11
7.2 Q6.2 . . . 11
7.3 Q6.3 . . . 11
7.4 Q6.4 . . . 11
7.5 Q6.5 . . . 11
7.6 Q6.6 . . . 11
7.7 Q6.7 . . . 11
7.8 Q6.8 . . . 12

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb

Table 1: Nomenclature Table

1 Assignment description

The first assignment of Modelling and Control of Manipulators focuses on the geometric fundamentals and algorithmic tools underlying any robotics application. The concepts of transformation matrix, orientation matrix and the equivalent representations of orientation matrices (Equivalent angle-axis representation and Euler Angles) will be reviewed.

The first assignment is **mandatory** and consists of 5 different exercises. You are asked to:

- Download the .zip file called MCM-LAB1 from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the predefined files called "*main.m*", "*AngleAxisToRot.m*", "*RotToAngleAxis.m*", "*YPRToRot.m*", "*RotToYPR.m*" and "*IsRotationMatrix.m*".
- Write a report motivating the answers for each exercise, following the predefind format on this document.
- The usage of built-in MATLAB functions is strictly forbidden except for basic mathematical operations such as *det*, *eig*.

1.1 Exercise 1 - Angle-Axis to Rotation Matrix

A particularly interesting minimal representation of 3D rotation matrices is the so-called angle-axis representation, where a rotation is represent by the axis of rotation **h** and the angle θ. Any rotation matrix can be represented by its equivalent angle-axis representation by applying the Rodrigues Formula.

Q1.1 Given an angle-axis pair $(\mathbf{h}, \theta)$, implement on MATLAB the Rodrigues formula, computing the equivalent rotation matrix, **WITHOUT** using built-in matlab functions. The function signature will be

function R = AngleAxisToRot(h,theta)

Then test it for the following cases and briefly comment the results obtained:

- **Q1.2** $\mathbf{h} = [1, 0, 0]^T$ and $\theta = 90°$
- **Q1.3** $\mathbf{h} = [0, 0, 1]^T$ and $\theta = \pi/3$
- **Q1.4** $\rho = [-\pi/3, -\pi/6, \pi/3]$;

1.2 Exercise 2 - Rotation Matrix to Angle-Axis

Given a rotation matrix R, the problem of finding the corresponding angle-axis representation $(\mathbf{h}, \theta)$ is called the Inverse Equivalent Angle-Axis Problem.

Q2.1 Given a rotation matrix R, implement on MATLAB the Equivalent Angle-Axis equations **WITHOUT** using built-in matlab functions. The function signature will be

function [h,theta] = RotToAngleAxis(R)

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix(R)*.
Hint: utilize a suitable tolerance (e.g. 10^{-3}) to check the properties of the matrices.
Test it for the following cases and briefly comment the results obtained:

- **Q2.2** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$
- **Q2.3** $R = \begin{pmatrix} 0.5 & -\sqrt{3}/2 & 0 \\ \sqrt{3}/2 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.4** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q2.5** $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

- **Q2.6** $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

1.3 Exercise 3 - Euler Angles to Rotation Matrix

Any orientation matrix can be expressed in terms of three elementary rotations in sequence. Consider the Yaw Pitch Roll (YPR) representation, where the sequence of the rotation axes is Z-Y-X.

Q3.1 Given a triplet of YPR angles (ψ, θ, ϕ), compute the equivalent rotation matrix representation **WITHOUT** using built-in matlab functions. The function signature will be

function R = YPRToRot(psi, theta, phi)

Then test it for the following cases and briefly comment the results obtained:

- **Q3.2** $\psi = \theta = 0$, $\phi = \pi/2$
- **Q3.3** $\phi = \theta = 0$, $\psi = 60°$
- **Q3.4** $\psi = \pi/3$, $\theta = \pi/2$, $\phi = \pi/4$
- **Q3.5** $\psi = 0$, $\theta = \pi/2$, $\phi = -\pi/12$

1.4 Exercise 4 - Rotation Matrix to Euler Angles

Given a rotation matrix R, it is possible to compute an equivalent triplet of YPR angles (ψ, θ, ϕ), provided that the configuration is not singular (that is, $\cos\theta \neq 0$).

Q4.1 Given a rotation matrix R, implement in MATLAB the equivalent YPR angles, **WITHOUT** using built-in matlab functions. The function signature will be

function [psi,theta,phi] = RotToYPR(R)

You **MUST** check that the input is a valid rotation matrix by filling in and utilizing the function *IsRotationMatrix(R)*. **Hint:** utilize a suitable tolerance (e.g.10^{-3}) to check the properties of the matrices.
Test it for the following cases and briefly comment the results obtained:

- **Q4.2** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$
- **Q4.3** $R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- **Q4.4** $R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0.5 & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$

1.5 Exercise 5 - Rotation Matrix to Angle-Axis using Eigenvectors

Consider a rotation matrix R and the corresponding angle-axis representation ($\mathbf{h}$, θ). Then, $R\mathbf{h} = \mathbf{h}$, which can be verified by computing $R\mathbf{h}$ with the Rodrigues Formula and observing that the terms containing $\mathbf{h} \times \mathbf{h}$ vanish. Therefore, $\mathbf{h}$ is an eigenvector of R corresponding to the eigenvalue +1.

Given the following rotation matrices, compute the corresponding angle-axis representation with two different methods. In the first method, compute $(\mathbf{h}, \theta)$ using the function *RotToAngleAxis* implemented before. In the second one, compute $\mathbf{h}$ as the eigenvector of R corresponding to eigenvalue +1, and then use the function *RotToAngleAxis* to compute θ. Compare and briefly discuss the results.

- **Q5.1** $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

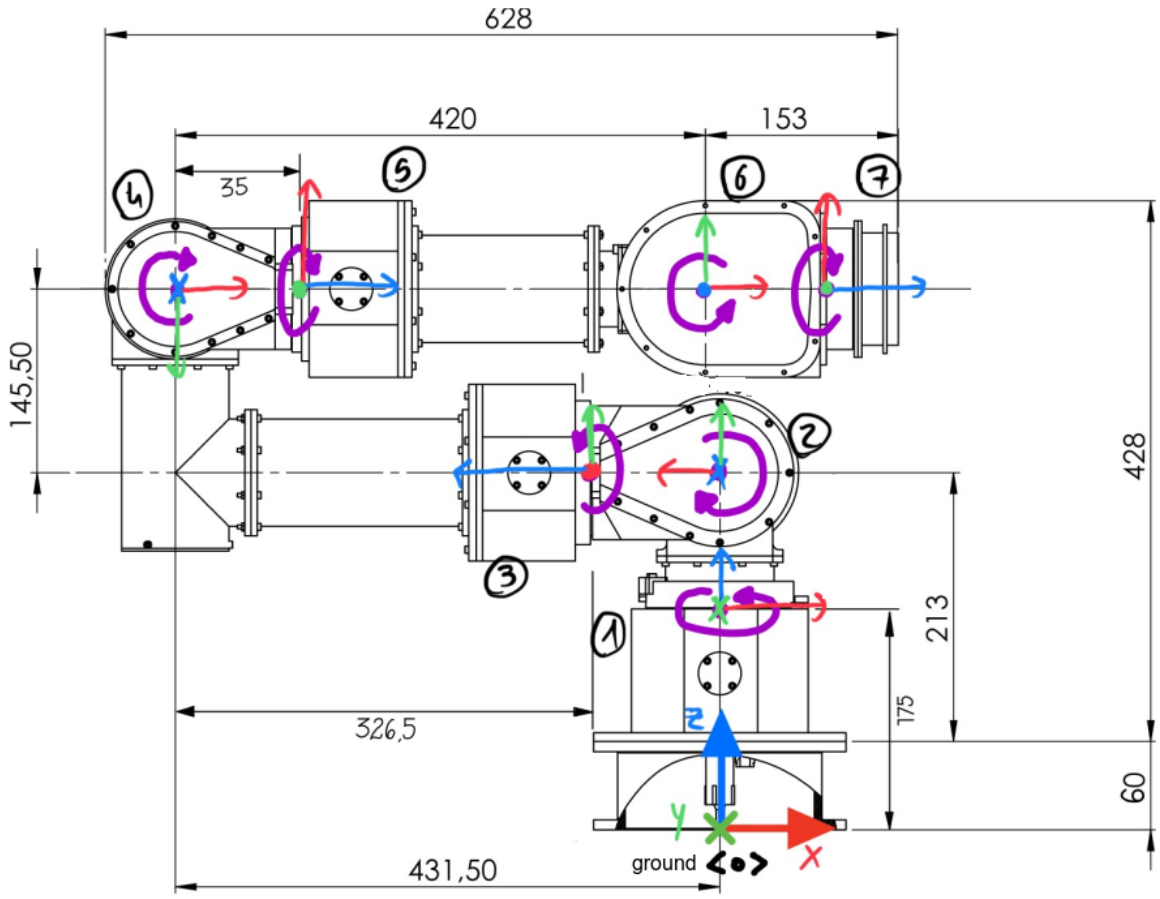


Figure 1: Exercise 6 frames. The unit of measurement is millimeters.

- **Q5.2** $R = \frac{1}{9}\begin{pmatrix} 4 & -4 & -7 \\ 8 & 1 & 4 \\ -1 & -8 & 4 \end{pmatrix}$

1.6 Exercise 6 - Frame tree

Figure 1 shows the frame tree for the 7 joints of the Franka robot. With reference to the figure, use the geometric definition of the transformation matrix to compute by hand the following matrices.

- **Q6.1** ${}^{0}_{1}T$
- **Q6.2** ${}^{1}_{2}T$
- **Q6.3** ${}^{2}_{3}T$
- **Q6.4** ${}^{3}_{4}T$
- **Q6.5** ${}^{4}_{5}T$
- **Q6.6** ${}^{5}_{6}T$
- **Q6.7** ${}^{6}_{6}T$
- **Q6.8** ${}^{7}_{6}T$

You **MUST** compute the matrices **WITHOUT** using mathematical software.

2 Exercise 1

Rotations can be represented in multiple ways; therefore, it is useful to implement functions that allow conversion between different representations. In this first exercise, the focus is on converting from the angle–axis representation to the rotation matrix.

2.1 Q1.1

The MATLAB function *AngleAxisToRot* requires two inputs: the rotation axis $\underline{h}$ and the angle of rotation θ (in radians). The output is the rotation matrix corresponding to the inputs and, in order to obtain it, an implementation of Rodrigues formula is used:

$$R = I + sin\theta[\underline{h}x] + (1 - cos\theta)[\underline{h}x]^2$$

To use this formula, the function computes the skew-symmetric operator.

2.2 Q1.2

Input: $\underline{h} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \theta = 90°$

Output: $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

The angle is in degrees and needs to be converted to radians. The inputs describe a rotation of 90° around x axis. Indeed, from the output matrix, it is possible to see that x remains unchanged, y axis corresponds to the previous z axis, and the new z axis is oriented opposite to the original y axis.

2.3 Q1.3

Input: $\underline{h} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \theta = \frac{\pi}{3}$

Output: $R = \begin{pmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Now the rotation is about the z axis and, from the rotation matrix, it is possible to see that z remains the same. The new x axis has positive components in both x and y directions, while the new y axis has negative x and positive y components. The values 0.866 and 0.5 correspond, respectively, to $sin(\pi/3)$ and $cos(\pi/3)$.

2.4 Q1.4

Input: $\underline{\rho} = \begin{pmatrix} -\frac{\pi}{3} \\ -\frac{\pi}{6} \\ \frac{\pi}{3} \end{pmatrix}$

Output: $R = \begin{pmatrix} 0.444 & -0.444 & -0.778 \\ 0.889 & 0.111 & 0.444 \\ -0.111 & -0.889 & 0.444 \end{pmatrix}$

In this case, the input is the rotation vector $\underline{\rho}$. To extract the rotation axis and angle, $\underline{\rho}$ must be normalized. The rotation angle θ is given by the norm of $\underline{\rho}$, and the unit rotation axis is obtained by dividing $\underline{\rho}$ by θ. The angle-axis parameters correspond to $\underline{h} = [-0.667, -0.333, 0.667]^T$ and $\theta = 90°$
The sign of the coordinates is consistent with the direction of rotation about the $\underline{h}$ axis.

3 Exercise 2

The Inverse Equivalent angle axis Problem involves finding the angle axis and the angle of rotation, theta, starting from the rotation matrix.

3.1 Q2.1

The *RotToAngleAxis* function accepts a rotation matrix as input and provides the corresponding angle-axis representation (angle θ and axis vector $\underline{h}$) as output.

Initially, the function performs validation to determine whether the input matrix constitutes a valid rotation matrix. If this condition is not met, the function alerts the user and terminates execution. If the matrix is confirmed to be a valid rotation matrix, the function proceeds to calculate the rotation angle theta using the formula $Tr(R) = 1 + 2cos\theta$.

Specific handling is implemented for singularity cases:

- If $\theta = 0°$, then $\underline{h}$ is arbitrary. The function therefore defaults to $\underline{h} = [1, 0, 0]^T$ and issues a warning to the user regarding this indeterminacy.
- If $\theta = 180°$, two possible axes with opposite directions exist. The function notifies the user of this ambiguity and selects as output the one that possesses a positive component along the x axis, while the other is printed in the console.

3.2 Q2.2

Input: $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

Output: $\underline{h} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \theta = 90°$

This case is the inverse of Q1.2; consequently, the output obtained is identical to the input used in that exercise.

3.3 Q2.3

Input: $R = \begin{pmatrix} 0.5 & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Output: $\underline{h} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \theta = 60°$

In this case, the rotation is about the z axis. From the rotation matrix, it is possible to see the values of $sin60°$ and $cos60°$.

3.4 Q2.4

Input: $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Output: $\underline{h} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \theta = 0°$

Since $R = I$, there is no actual rotation ($\theta = 0°$) so the function indicates that $\underline{h}$ is arbitrary.

3.5 Q2.5

Input: $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Output: $\underline{h} = \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix}, \quad \theta = 180°$

The given matrix corresponds to a rotation of π radians around z. For this rotation, two equivalent angle–axis representations exist, thus, the function outputs one of them as the result and displays the alternative representation in the console, which is: $\underline{h} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$.

3.6 Q2.6

Input: $R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Output: The matrix is not a rotation matrix.

The input matrix transforms a right-handed reference frame into a left-handed one. Consequently, it is not a proper rotation matrix, and the function notifies the user before terminating.

4 Exercise 3

By using the Euler Theorem it can be observed that any 3D frame in the Euclidean space can be rotated about a single fixed axis, just by considering a single rotation of an angle theta $[0, \pi]$, in order to align it to another arbitrary fixed frame. Moreover, Euler Angles can be used as a parametrization method for a Special Orthogonal group in 3 dimensions (also known as SO(3)), as it was done in this exercise. In fact, a Rotation matrix R was created out of 3 angles (ψ, θ, ϕ) taking the "yaw, pitch, roll" rotations into account, which respectively describe rotations around (z, y, x) axes.

4.1 Q3.1

Having the YPR angles given in four different cases, the function R = YPRToRot (ψ, θ, ϕ) was implemented, which computes the rotation matrices in this way:

$$R = \begin{pmatrix} \cos\psi\cos\theta & \cos\psi\sin\theta\sin\phi - \sin\psi\cos\phi & \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi \\ \sin\psi\cos\theta & \sin\psi\sin\theta\sin\phi + \cos\psi\cos\phi & \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi \\ -\sin\theta & \cos\theta\sin\phi & \cos\theta\cos\phi \end{pmatrix}$$

However, $\theta = \pi/2$ (the robot gets a total pitch of $90°$) in two of the four cases, so that this gives a singularity circumstance and the inverse mapping is no longer unique. This does not let the roll and yaw rotations be unique anymore, creating error situations where there actually are not. In fact, two different roll and yaw combinations produce the same orientation, so that a degree of freedom is lost. This does not allow to describe all the angular velocities because some rotation directions can not be managed properly with YPR angles.

4.2 Q3.2

Input: $\psi = \theta = 0, \quad \phi = \pi/2$

Output: $R3_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

Here, the result is a rotation matrix which execute a rotation about the x axis (the only angle different from 0 is ϕ). A $90°$ rotation around the x axis (roll) puts y in the z position and z in the $-y$ position.

4.3 Q3.3

Input: $\phi = \theta = 0, \quad \psi = 60°$

Output: $R3_3 = \begin{pmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

In this case, the rotation matrix represents a rotation about z, with ψ as the only nonzero angle. The $60°$ rotation moves the x axis toward y, while the y axis is rotated toward the $-x$ direction within the xy plane.

4.4 Q3.4 - Q3.5

$$R3_4 = \begin{pmatrix} 0 & -0.259 & 0.966 \\ 0 & 0.966 & 0.259 \\ -1 & 0 & 0 \end{pmatrix}$$

$$R3_5 = \begin{pmatrix} 0 & -0.259 & 0.966 \\ 0 & 0.966 & 0.259 \\ -1 & 0 & 0 \end{pmatrix}$$

For the last two cases, the input configuration (containing a singularity condition because $cos\theta = 0$) shows that the orientation depends only on the ($\phi - \psi$) relation and not on those angles taken individually. As previously stated, the yaw and roll are merged into a singular rotation. In fact, if ϕ and ψ are changed but their difference remains the same, the rotation matrix does not change.

5 Exercise 4

If it is possible to compute rotational matrices by having YPR rotations, as shown in the previous exercise, it is also possible to do the contrary.

5.1 Q4.1

In fact, in this exercise it is asked to compute YPR angles having 3 rotational matrices given by using the function $[\psi, \theta, \phi] = RotToYPR(R)$. It is mandatory to not let $cos\theta$ be equal to zero ($\theta = 90°$) in order to not have singularity configurations, while also checking the properties of the matrices by using a suitable tolerance (10^{-3} in this case). A check is also implemented to observe whether the input matrix is rotational or not, by using the function IsRotationMatrix(R) implemented before.

The 3 angles (θ, ψ, ϕ) were computed as follows:

$\theta = atan2(-R_{31}, \sqrt{R_{11}^2 + R_{21}^2})$
$\psi = atan2(R_{21}, R_{11})$
$\phi = atan2(R_{32}, R_{33})$

5.2 Q4.2

Input: $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

Output: $\psi = 0, \quad \theta = 0, \quad \phi = 1.5708$

This shows that y axis rotates towards z and the z axis towards $-y$. This is a $90°$ roll.

5.3 Q4.3

Input: $R = \begin{pmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} & 0 \\ \frac{\sqrt{3}}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$

Output: $\psi = 1.0472, \quad \theta = 0, \quad \phi = 0$

Pure $60°$ rotation in the XY plane. This is a typical rotation around the z axis.

5.4 Q4.4

Input: $R = \begin{pmatrix} 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{1}{2} & \frac{\sqrt{2}\sqrt{3}}{4} & \frac{\sqrt{2}\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{2} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \end{pmatrix}$

Output: $\psi = 1.5708, \quad \theta = 1.0472, \quad \phi = 0.7854$

This is a combined rotation which makes the system tilt $60°$ (pitch) and then rotates it $45°$ around the new x axis, after a $90°$ initial rotation around the z axis. $\theta = \pi/3$, it is near the singularity condition.

6 Exercise 5

In this exercise, the goal consists of finding the angle-axis representation $(\underline{h}, \theta)$ of a rotation matrix R, first using the function already implemented *RotToAngleAxis*, and then computing $\underline{h}$ as the eigenvector of R corresponding to the eigenvalue +1. Since *RotToAngleAxis* has already been explained in exercise 2, only the implementation of the second method will be discussed. The eigenvalue decomposition of R was computed with

```
[V, D] = eig(R);
```

where D is the diagonal matrix containing the eigenvalues of R and the columns of V are the corresponding eigenvectors. Since the rotation axis corresponds to the eigenvalue +1, the code

```
idx = find(abs(diag(D) - 1) < 1e-3);
h = V(:, idx);
h = h / norm(h);
```

selects the eigenvector whose eigenvalue is approximately 1 (within a numerical tolerance of 10^{-3}) and normalizes it to obtain the unit vector $\underline{h}$.

6.1 Q5.1

Input: $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

Output: $h = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \theta = 90°$

After running the MATLAB script to compute $\underline{h}$, the output is the same as that for both methods:

6.2 Q5.2

Input: $R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$

Output from *RotToAngleAxis*: $\underline{h} = \begin{pmatrix} -0.6667 \\ -0.3333 \\ 0.6667 \end{pmatrix}, \quad \theta = 90°$

Output from eigenvalue decomposition: $\underline{h} = \begin{pmatrix} 0.6667 \\ 0.3333 \\ -0.6667 \end{pmatrix}, \quad \theta = 90°$

In this case, the two methods give an equal and opposite result for $\underline{h}$. This is due to the fact that the eigenvector associated with the eigenvalue +1 defines the rotation axis orientation but not its direction, since eigenvectors are determined up to a sign. For this reason, the eigenvalue method alone is not sufficient to fully recover the angle–axis representation. Thus, the function RotToAngleAxis is required to compute both the correct sign of the axis and the corresponding rotation angle.

7 Exercise 6

A transformation matrix ${}^a_bT$, which expresses frame $< b >$ with respect to frame $< a >$, is defined as

$$ {}^a_bT = \begin{pmatrix} {}^a_bR & {}^aOb \\ 0\,0\,0 & 1 \end{pmatrix} $$

where:

- ${}^a_bR$ is the rotation matrix of frame $< b >$ with respect to frame $< a >$;
- ${}^aO_b$ is the position vector of the origin of $< b >$ expressed in $< a >$.

Looking at Figure 1, we can obtain the following transformation matrices.

7.1 Q6.1

$$ {}^0_1T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 175 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.2 Q6.2

$$ {}^1_2T = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 98 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.3 Q6.3

$$ {}^2_3T = \begin{pmatrix} 0 & 0 & 1 & 105 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.4 Q6.4

$$ {}^3_4T = \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 145.5 \\ -1 & 0 & 0 & 326.5 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.5 Q6.5

$$ {}^4_5T = \begin{pmatrix} 0 & 0 & 1 & 35 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.6 Q6.6

$$ {}^5_6T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 385 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.7 Q6.7

$$ {}^6_6T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

7.8 Q6.8

$$ {}^{7}_{6}T = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & -z \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

Note: the value $-z$ in the last matrix represents a negative value along the referring z axis, due to the lack of distance shares between frames $< 6 >$ and $< 7 >$.